

Estudio Computacional de $A(n, d)$: El Máximo Tamaño de un Código Binario con Longitud n y Distancia Mínima d

Amador Jesus, Hernández Roberto, Hernández Belinda, Ramos Sarai

May 24, 2026

Abstract

El número $A(n, d)$ representa la máxima cardinalidad posible de un código binario de longitud n y distancia mínima de Hamming d . Este problema, central en la teoría de códigos correctores de errores, tiene aplicaciones en comunicaciones digitales y almacenamiento de datos. En este trabajo presentamos un enfoque computacional para determinar $A(n, d)$ para valores pequeños de n y d . Reformulamos el problema como la búsqueda de una clique máxima en un grafo de Hamming, implementamos algoritmos exactos (Bron–Kerbosch) y exploramos cotas superiores mediante programación lineal. El reporte incluye el marco teórico, la metodología algorítmica, el diseño de la implementación en Python y los resultados esperados. Todo el código se encuentra en un repositorio de GitHub para facilitar la colaboración y la reproducibilidad.

1 Introducción

Un *código binario* $C \subseteq \mathbb{F}_2^n$ es un conjunto de palabras (vectores) de longitud n sobre el alfabeto $\{0, 1\}$. La *distancia de Hamming* $d_H(x, y)$ entre dos vectores $x, y \in \mathbb{F}_2^n$ es el número de coordenadas en las que difieren. La *distancia mínima* del código es $d = \min_{x \neq y \in C} d_H(x, y)$. El parámetro $A(n, d)$ se define como

$$A(n, d) = \max\{|C| : C \subseteq \mathbb{F}_2^n, \forall x \neq y \in C, d_H(x, y) \geq d\}.$$

Determinar $A(n, d)$ es un problema combinatorio de gran relevancia porque permite conocer el tamaño máximo de un código que puede detectar o corregir un cierto número de errores. Dado que el problema es $\mathcal{NP}$ -duro (para d fijo y n variable), no existe una fórmula cerrada general. Sin embargo, para valores pequeños de n y d se han tabulado valores exactos o cotas ajustadas mediante búsqueda exhaustiva y métodos algebraicos.

En este proyecto implementamos un paquete en Python que:

1. Genera el grafo de Hamming asociado a (n, d) ,

2. Calcula el número clique máximo (solución exacta para n pequeños) mediante el algoritmo de Bron–Kerbosch,
3. Proporciona cotas inferiores heurísticas y cotas superiores mediante programación lineal (cota de Delsarte),
4. Valida los resultados con tablas de referencia (Brouwer, Agrell).

El presente reporte describe los fundamentos teóricos, la metodología computacional y el plan de implementación. Todo el código fuente está versionado en GitHub, lo que permite un trabajo colaborativo eficiente entre los miembros del equipo.

2 Distancia de Hamming

Si $x, y \in \{0, 1\}^n = \{0, 1\} \times \{0, 1\} \times \cdots \times \{0, 1\}$, entonces $\text{dis}(x, y)$ denota la distancia de Hamming entre x e y (es decir, la cantidad de símbolos distintos). Un código de longitud n y distancia mínima d es un subconjunto $C \subseteq \{0, 1\}^n$ tal que $\text{dis}(x, y) \geq d$ para todo $x, y \in C$. Denotemos por $A(n, d)$ el número máximo de n -tuplas en un código de longitud n con distancia mínima d .

3 Marco Teórico

3.1 Reducción a Problema de Clique en un Grafo

Definimos el *grafo de Hamming* $H(n, d) = (V, E)$ donde:

$$V = \mathbb{F}_2^n = \{0, 1\}^n, \quad E = \{\{u, v\} \subset V : u \neq v, d_H(u, v) \geq d\}.$$

Entonces, un código C con distancia mínima $\geq d$ corresponde exactamente a un *conjunto independiente* en el complemento del grafo, o equivalentemente a una *clique* en $H(n, d)$. Por lo tanto,

$$A(n, d) = \omega(H(n, d)),$$

donde $\omega(G)$ denota el número de la clique máxima (el mayor conjunto de vértices mutuamente adyacentes). Esta reformulación permite usar algoritmos estándar de teoría de grafos.

3.2 Cotas Fundamentales

Para acotar $A(n, d)$ sin necesidad de una búsqueda exhaustiva, se utilizan desigualdades clásicas:

- **Cota de Hamming (esfera-empaquetamiento):**

$$A(n, d) \leq \frac{2^n}{\sum_{i=0}^{\lfloor (d-1)/2 \rfloor} \binom{n}{i}}.$$

- **Cota de Singleton:**

$$A(n, d) \leq 2^{n-d+1}.$$

- **Cota de Plotkin:** si $d > n/2$, entonces $A(n, d) \leq \lfloor \frac{2d}{2d-n} \rfloor$.
- **Cota de Gilbert–Varshamov (existencial):** existe un código de tamaño al menos $\frac{2^n}{\sum_{i=0}^{d-1} \binom{n}{i}}$, por lo que $A(n, d) \geq$ esa cantidad.

Estas cotas nos proporcionan intervalos iniciales y son útiles para verificar la corrección de los resultados computacionales.

3.3 Complejidad Computacional

El problema de determinar $\omega(G)$ para un grafo general es $\mathcal{NP}$ -duro y, bajo la hipótesis $\mathcal{P} \neq \mathcal{NP}$, no admite un algoritmo polinomial. Para $H(n, d)$, el número de vértices es 2^n , lo que restringe los casos tratables exactamente a $n \leq 10$ o $n \leq 12$ (dependiendo de d). Para valores mayores, debemos recurrir a cotas superiores e inferiores obtenidas por métodos heurísticos o de optimización.

4 Metodología Computacional

4.1 Generación del Grafo de Hamming

La generación explícita de la matriz de adyacencia requiere $O(2^{2n})$ comparaciones de distancia. Para $n \leq 10$ esto es factible (poco más de 10^6 pares). Implementamos una función que:

- Enumera todos los vectores de $\mathbb{F}_2^n$ como enteros de 0 a $2^n - 1$ (representación binaria).
- Calcula la distancia de Hamming mediante la operación XOR y el conteo de bits (función `popcount`).
- Construye un grafo en memoria usando listas de adyacencia (para eficiencia).

Observación sobre la escalabilidad computacional: Es crucial notar el crecimiento exponencial del grafo de Hamming $H(n, d)$ al variar sus parámetros. Al aumentar la longitud del código n en una sola unidad, el número total de vértices en el espacio de búsqueda se duplica, pasando de 2^n a 2^{n+1} . Consecuentemente, el número de aristas a evaluar e insertar en el grafo crece de forma drástica. Este rápido aumento en el tamaño de la gráfica representa el principal cuello de botella computacional del problema.

4.2 Algoritmos Exactos: Bron–Kerbosch

El algoritmo de Bron–Kerbosch con pivoteo encuentra todas las cliques maximales de un grafo. Su complejidad en el peor caso es $O(3^{|V|/3})$, pero en la práctica es muy eficiente para grafos dispersos. Utilizamos una versión recursiva que mantiene tres conjuntos:

- R: clique en construcción,
- P: candidatos que pueden extender la clique,

- **X**: vértices ya procesados (evita duplicados).

El pivote se elige de $P \cup X$ para minimizar las ramificaciones. El algoritmo se detiene cuando P y X están vacíos, registrando R como una clique maximal. El tamaño máximo entre todas las cliques maximales es $\omega(G)$.

Para $n = 10$, $|V| = 1024$, el grafo suele ser denso cuando d es pequeño, lo que puede provocar explosión combinatoria. En esos casos se aplican podas adicionales (por ejemplo, ordenar los vértices por grado degenerado).

4.3 Enfoques Heurísticos para n Moderadamente Grande

Para $n > 12$ no es factible ejecutar Bron–Kerbosch. Proponemos dos heurísticas:

1. **Búsqueda local greedy**: se parte de un vértice aleatorio y se añaden iterativamente los vértices que sean adyacentes a todos los ya seleccionados, priorizando aquellos con mayor grado.
2. **Algoritmo de recocido simulado**: se define una función de energía negativa del tamaño de la clique y se aplican movimientos de intercambio de vértices.

Estas heurísticas proporcionan cotas inferiores (códigos grandes) que pueden ser mejoradas con el tiempo de ejecución.

4.4 Cotas Superiores mediante Programación Lineal

La cota de Delsarte [4], también conocida como cota de programación lineal, ha sido históricamente la mejor para muchos valores de n y d . Se basa en las desigualdades de MacWilliams y en los polinomios de Krawtchouk. Resolvemos el siguiente programa lineal (dual) para obtener una cota superior:

$$\max \sum_{i=0}^n \beta_i A_i \quad \text{sujeto a} \quad \beta_0 = 1, \beta_i \geq 0, \sum_{i=0}^n \beta_i K_i(j) \geq 0 \quad \forall j,$$

donde $K_i(j)$ es el polinomio de Krawtchouk y (A_i) es la distribución de distancias de un código. Este programa se resuelve con bibliotecas de optimización lineal como `pulp` o `ortools`.

4.5 Estrategia Combinada: Acotamiento de $A(n, d)$

Dado que los métodos heurísticos no garantizan optimalidad global, nuestro paquete computacional combina los resultados empíricos con límites teóricos para acotar el valor exacto de $A(n, d)$.

La estrategia computacional define un intervalo de la forma:

$$A_{\text{inf}}(n, d) \leq A(n, d) \leq A_{\text{sup}}(n, d)$$

- **Cota Inferior (A_{inf})**: Se obtiene mediante el tamaño máximo del código generado por nuestro algoritmo goloso multiexploratorio. Toda solución válida construida demuestra la existencia de un código de ese tamaño.

- **Cota Superior (A_{sup}):** Se calcula dinámicamente evaluando la cota de empaquetamiento estricto de esferas (Cota de Hamming) y la cota de Plotkin. El sistema selecciona automáticamente el valor mínimo (el más restrictivo) entre ambas cotas para el par (n, d) evaluado.

5 Plan de Implementación y Resultados Esperados

5.1 Validación con Tablas Conocidas

Compararemos los resultados de nuestro código con las tablas de referencia de Brouwer [3] y Agrell [2]. Por ejemplo, sabemos que:

$$A(5, 3) = 4, \quad A(7, 3) = 16, \quad A(8, 3) = 20.$$

Nuestro algoritmo exacto debe reproducir estos valores para $n \leq 8$. Para $n = 9, 10$ compararemos las cotas obtenidas por programación lineal con las reportadas en la literatura.

Table 1: Tabla actualizada de cotas para códigos binarios de longitud menor a 25, con extensión para $d > 10$

(Updated Table of Bounds for Binary Codes of Length Less than 25, with Extension for $d > 10$)

Descripción: Esta tabla muestra las mejores cotas superiores e inferiores conocidas (rangos separados por guiones) o valores exactos para códigos binarios de longitud n (filas) y distancia mínima d (columnas). Los valores únicos indican cotas exactas. Los rangos como $a - b$ representan el intervalo entre la cota inferior conocida a y la cota superior conocida b . Para $n \geq 25$, se incluyen expresiones como $2^{19} - 599184$ que denotan valores exactos o cotas calculadas. Esta tabla es una extensión de resultados clásicos para distancias $d > 10$, actualizada hasta $n = 28$.

n	$d = 4$	$d = 6$	$d = 8$	$d = 10$	$d = 12$	$d = 14$	$d = 16$
6	4	2	1	1	1	1	1
7	8	2	1	1	1	1	1
8	16	2	2	1	1	1	1
9	20	4	2	1	1	1	1
10	40	6	2	2	1	1	1
11	72	12	2	2	1	1	1
12	144	24	4	2	2	1	1
13	256	32	4	2	2	1	1
14	512	64	8	2	2	2	1
15	1024	128	16	4	2	2	1
16	2048	256	32	4	2	2	2
17	2816-3276	258-340	36	6	2	2	2
18	5632-6552	512-673	64	10	4	2	2
19	10496-13104	1024-1237	128	20	4	2	2
20	20480-26168	2048-2279	256	40	6	2	2
21	40960-43688	2560-4096	512	42-47	8	4	2
22	81920-87333	4096-6941	1024	64-84	12	4	2
23	163840-172361	8192-13674	2048	80-150	24	4	2
24	327680-344308	16384-24106	4096	136-268	48	6	4
25	$2^{19} - 599184$	17920-47538	4096-5421	192-466	52-55	8	4
26	$2^{20} - 1198368$	32768-84260	4104-9275	384-836	64-96	14	4
27	$2^{21} - 2396736$	65536-157285	8192-17099	512-1585	128-169	28	6
28	$2^{22} - 4792950$	131072-291269	16384-32151	1024-2817	178-288	56	8

5.2 Resultados Esperados y Análisis

Se espera que para $n \leq 10$ y d moderado ($d \geq 3$) podamos obtener el valor exacto de $A(n, d)$. Para casos más allá de la capacidad exacta, presentaremos intervalos:

$$A_{\text{inf}}(n, d) \leq A(n, d) \leq A_{\text{sup}}(n, d),$$

donde A_{inf} proviene de las heurísticas y A_{sup} de la cota de Delsarte. Además, mostraremos gráficas de la evolución del tamaño máximo en función de n para d fijo.

6 Conclusiones y Trabajo Futuro

En este trabajo hemos presentado una metodología computacional para abordar el problema de determinar $A(n, d)$. La reducción a problema de clique en el grafo de Hamming permite aplicar algoritmos bien estudiados, y el uso de cotas teóricas complementa los resultados exactos cuando la búsqueda exhaustiva es inviable. El paquete Python desarrollado será una herramienta didáctica y de referencia para el estudio de códigos binarios.

Como trabajo futuro, planeamos:

- Incorporar la cota de programación semidefinida (SDP) descrita por Schrijver [5], que mejora la cota de Delsarte.
- Paralelizar el algoritmo de Bron–Kerbosch para aprovechar múltiples núcleos.
- Extender el estudio a códigos no binarios (alfabetos de tamaño q).

Todo el código y el presente reporte están disponibles en el repositorio GitHub del equipo, favoreciendo la transparencia y la colaboración.

References

- [1] M. R. Best, A. E. Brouwer, F. J. MacWilliams, A. M. Odlyzko, N. J. A. Sloane. Bounds for binary codes of length less than 25. *IEEE Trans. Inf. Theory*, IT-24(1):81–93, 1978.
- [2] E. Agrell, A. Vardy, K. Zeger. A table of upper bounds for binary codes. *IEEE Trans. Inf. Theory*, 47(7):3004–3006, 2001.
- [3] A. E. Brouwer. Bounds for binary codes. Tablas en línea: <https://www.win.tue.nl/~aeb/codes/binary-1.html>, 2018.
- [4] P. Delsarte. An algebraic approach to the association schemes of coding theory. *Philips Research Reports Supplements*, No. 10, 1973.
- [5] A. Schrijver. New code upper bounds from the Terwilliger algebra and semidefinite programming. *IEEE Trans. Inf. Theory*, 51(8):2859–2866, 2005.
- [6] C. Bron, J. Kerbosch. Algorithm 457: Finding All Cliques of an Undirected Graph. *Commun. ACM*, 16(9):575–577, 1973.
- [7] E. Tomita, A. Tanaka, H. Takahashi. The worst-case time complexity for generating all maximal cliques. *Theor. Comput. Sci.*, 363(3):260–267, 2006.
- [8] P. R. J. Östergård. A fast algorithm for the maximum clique problem. *Discrete Applied Mathematics*, 120(1-3):197–207, 2002.
- [9] Sage Developers. *SageMath Documentation: Coding Theory*. <https://doc.sagemath.org/html/en/reference/coding/>, 2024.

A Demostraciones de Cotas Teóricas

En esta sección se presentan las demostraciones formales de las cotas superiores utilizadas en la herramienta computacional para limitar el espacio de búsqueda del parámetro $A(n, d)$.

[12pt, spanish]article [utf8]inputenc [T1]fontenc [spanish]babel graphicx float array booktabs geometry fancyhdr setspace parskip amsmath amsthm amsfonts amssymb xcolor listings caption subcaption caption hyperref geometry margin=1in tikz eso-pic [pages=some]background

Pruebas de las Cotas Fundamentales

Cota de Hamming (esfera-empaquetamiento)

Sea $t = \lfloor \frac{d-1}{2} \rfloor$. Entonces

$$A(n, d) \leq \frac{2^n}{\sum_{i=0}^t \binom{n}{i}}.$$

Proof. Sea $C \subseteq \{0, 1\}^n$ un código con distancia mínima d y $|C| = M$. Para cada $c \in C$ definimos la *bola cerrada* de radio t :

$$B(c, t) = \{x \in \{0, 1\}^n : d_H(x, c) \leq t\}.$$

Afirmamos que estas bolas son **disjuntas**. En efecto, supongamos que existe $x \in B(c, t) \cap B(c', t)$ con $c \neq c'$. Entonces, por la desigualdad triangular de la distancia de Hamming,

$$d_H(c, c') \leq d_H(c, x) + d_H(x, c') \leq t + t = 2t \leq d - 1 < d,$$

lo cual contradice que d es la distancia mínima de C .

Como cada bola contiene exactamente $\sum_{i=0}^t \binom{n}{i}$ elementos (los vectores que difieren de c en exactamente i coordenadas, $0 \leq i \leq t$), y todas están contenidas en $\{0, 1\}^n$, la condición de disjunción implica

$$M \cdot \sum_{i=0}^t \binom{n}{i} \leq |\{0, 1\}^n| = 2^n.$$

Despejando M obtenemos el resultado. □

Cota de Singleton

$$A(n, d) \leq 2^{n-d+1}.$$

Proof. Sea C un código con distancia mínima d . Considera la proyección $\pi : \{0, 1\}^n \rightarrow \{0, 1\}^{n-d+1}$ que retiene únicamente las primeras $n - d + 1$ coordenadas. Si $x, y \in C$ con $x \neq y$ satisficieran $\pi(x) = \pi(y)$, entonces x e y coincidirían en las primeras $n - d + 1$ coordenadas y, por lo tanto, sólo podrían diferir en las restantes $d - 1$ posiciones, dando $d_H(x, y) \leq d - 1 < d$, contradicción. Luego $\pi|_C$ es inyectiva, y como $|\{0, 1\}^{n-d+1}| = 2^{n-d+1}$, se sigue $|C| \leq 2^{n-d+1}$. □

Cota de Plotkin

Si $2d > n$, entonces $A(n, d) \leq \left\lfloor \frac{2d}{2d-n} \right\rfloor$.

Proof. Sea $C = \{c_1, \dots, c_M\}$. Contamos la suma de todas las distancias entre pares:

$$S = \sum_{1 \leq i < j \leq M} d_H(c_i, c_j).$$

Cota inferior de S . Como $d_H(c_i, c_j) \geq d$ para todo par $i \neq j$:

$$S \geq \binom{M}{2} d = \frac{M(M-1)}{2} d.$$

Cota superior de S . Para cada coordenada $k \in \{1, \dots, n\}$, sea n_k la cantidad de palabras de C con un 1 en la posición k . La contribución de la coordenada k a S es $n_k(M - n_k)$. Usando $n_k(M - n_k) \leq M^2/4$:

$$S = \sum_{k=1}^n n_k(M - n_k) \leq \frac{nM^2}{4}.$$

Combinando.

$$\frac{M(M-1)}{2} d \leq \frac{nM^2}{4} \implies 2d(M-1) \leq nM \implies (2d-n)M \leq 2d.$$

Como $2d > n$ se tiene $2d - n > 0$, así que $M \leq \frac{2d}{2d-n}$. Tomando parte entera: $A(n, d) \leq \left\lfloor \frac{2d}{2d-n} \right\rfloor$. □

Cota de Gilbert–Varshamov (cota inferior)

$$A(n, d) \geq \frac{2^n}{\sum_{i=0}^{d-1} \binom{n}{i}}.$$

Proof. Construimos C de manera *greedy*: partiendo de $C = \emptyset$, en cada paso agregamos a C una palabra de $\{0, 1\}^n$ que tenga distancia $\geq d$ con todas las palabras ya incluidas. El proceso termina cuando ninguna palabra restante puede ser añadida, es decir, cuando cada $x \in \{0, 1\}^n \setminus C$ cumple $d_H(x, c) < d$ para algún $c \in C$; en otras palabras, $x \in B(c, d-1)$ para algún $c \in C$. Esto significa

$$\{0, 1\}^n \subseteq \bigcup_{c \in C} B(c, d-1),$$

y tomando cardinalidades:

$$2^n \leq |C| \cdot \sum_{i=0}^{d-1} \binom{n}{i}.$$

Despejando obtenemos $|C| \geq \frac{2^n}{\sum_{i=0}^{d-1} \binom{n}{i}}$, y como C es un código válido, $A(n, d)$ es al menos tan grande. □